
Capstone Project 1

MongoDB Capstone Project

Online Store Order Analytics

Database

```
use("online_store_db")
```

Collections:

- customers
 - products
 - orders
-

1.Create Customers Collection

```
db.customers.insertMany([

{ customer_id: 1, name: "Aarav", city: "Hyderabad", membership: "Gold",
age: 24 },

{ customer_id: 2, name: "Priya", city: "Bangalore", membership: "Silver",
age: 28 },

{ customer_id: 3, name: "Rahul", city: "Mumbai", membership: "Gold", age:
32 },

{ customer_id: 4, name: "Sneha", city: "Delhi", membership: "Silver", age:
26 },

{ customer_id: 5, name: "Kiran", city: "Hyderabad", membership: "Gold",
age: 30 },
```

```
{ customer_id: 6, name: "Meera", city: "Chennai", membership: "Bronze",  
age: 27 }  
  
])
```

2. Create Products Collection

```
db.products.insertMany([  
  { product_id: 101, name: "Laptop", category: "Electronics", price: 75000,  
    stock: 10 },  
  { product_id: 102, name: "Phone", category: "Electronics", price: 50000,  
    stock: 15 },  
  { product_id: 103, name: "Desk", category: "Furniture", price: 15000,  
    stock: 8 },  
  { product_id: 104, name: "Chair", category: "Furniture", price: 7000,  
    stock: 20 },  
  { product_id: 105, name: "Tablet", category: "Electronics", price: 30000,  
    stock: 12 },  
  { product_id: 106, name: "Printer", category: "Electronics", price: 12000,  
    stock: 5 }  
])
```

3. Create Orders Collection

```
db.orders.insertMany([  
  { order_id: 1001, customer_id: 1, product_id: 101, quantity: 1,  
    order_date: "2026-03-01", status: "Delivered" },  
  { order_id: 1002, customer_id: 2, product_id: 102, quantity: 2,  
    order_date: "2026-03-02", status: "Delivered" },  
  { order_id: 1003, customer_id: 1, product_id: 105, quantity: 1,  
    order_date: "2026-03-03", status: "Pending" },  
  { order_id: 1004, customer_id: 3, product_id: 103, quantity: 1,  
    order_date: "2026-03-05", status: "Delivered" },  
  { order_id: 1005, customer_id: 5, product_id: 102, quantity: 3,  
    order_date: "2026-03-07", status: "Cancelled" },  
  { order_id: 1006, customer_id: 6, product_id: 104, quantity: 4,  
    order_date: "2026-03-08", status: "Delivered" },  
  { order_id: 1007, customer_id: 4, product_id: 106, quantity: 2,  
    order_date: "2026-03-09", status: "Pending" },  
  { order_id: 1008, customer_id: 3, product_id: 101, quantity: 1,  
    order_date: "2026-03-10", status: "Delivered" }  
])
```

Assignment Tasks

Basic Queries

. Display all documents from the customers collection.

db.customers.find();

. Display all documents from the products collection.

db.products.find();

. Display all documents from the orders collection.

db.orders.find();

. Display customers from Hyderabad.

db.customers.find({ city: "Hyderabad" });

. Display products from the Electronics category.

db.products.find({ category: "Electronics" });

. Display orders with status Delivered.

db.orders.find({ status: "Delivered" });

Filtering Queries

. Display products whose price is greater than 30000.

db.products.find({ price: { \$gt: 30000 } });

. Display products whose price is between 10000 and 50000.

db.products.find({ price: { \$gte: 10000, \$lte: 50000 } });

. Display customers whose age is greater than 26.

db.customers.find({ age: { \$gt: 26 } });

. Display orders whose quantity is greater than 1.

db.orders.find({ quantity: { \$gt: 1 } });

. Display products whose stock is less than or equal to 10.

db.products.find({ stock: { \$lte: 10 } });

. Display orders whose status is not Cancelled.

```
db.orders.find({ status: { $nt: "Cancelled" } });
```

. Display customers from Hyderabad or Mumbai.

```
db.customers.find({ city: { $in: ["Hyderabad", "Mumbai"] } });
```

Projection Queries

. Display only customer name and city.

```
db.customers.find({}, { name: 1, city: 1, _id: 0 });
```

. Display only product name, category, and price.

```
db.products.find({}, { name: 1, category: 1, price: 1, _id: 0 });
```

. Display only order_id, quantity, and status.

```
db.orders.find({}, { order_id: 1, quantity: 1, status: 1, _id: 0 });
```

Sorting, Limit, Skip

. Display products sorted by price ascending.

```
db.products.find().sort({ price: 1 });
```

. Display products sorted by price descending.

```
db.products.find().sort({ price: -1 });
```

. Display the top 3 most expensive products.

```
db.products.find().sort({ price: -1 }).limit(3);
```

. Display the 2 cheapest products.

```
db.products.find().sort({ price: 1 }).limit(2);
```

. Skip the first 2 products and display the rest.

```
db.products.find().skip(2);
```

. Display customers sorted by age descending.

```
db.customers.find().sort({ age: -1 })
```

Update Operations

- . Update the price of Laptop to 78000.

```
db.products.updateOne({ name: "Laptop" }, { $set: { price: 78000 } });
```

- . Add a new field discount: 10 to all Electronics products.

```
db.products.updateMany({ category: "Electronics" }, { $set: { discount: 10 } });
```

- . Add priority: "High" to orders with status Pending.

```
db.orders.updateMany({ status: "Pending" }, { $set: { priority: "High" } });
```

- . Change Meera's membership from Bronze to Silver.

```
db.customers.updateOne({ name: "Meera" }, { $set: { membership: "Silver" } });
```

Delete Operations

- . Delete the product named Printer.

```
db.products.deleteOne({ name: "Printer" })
```

- . Delete all products in the Furniture category.

```
db.products.deleteMany({ category: "Furniture" })
```

- . Delete orders whose status is Cancelled.

```
db.orders.deleteMany({ status: "Cancelled" })
```

Count Queries

- . Count total customers.

```
db.customers.countDocuments({});
```

- . Count products in the Electronics category.

```
db.products.countDocuments({ category: "Electronics" });
```

. Count delivered orders.

```
db.orders.countDocuments({ status: "Delivered" });
```

. Count customers from Hyderabad.

```
db.customers.countDocuments({ city: "Hyderabad"});
```

Aggregation Queries

. Find total stock available per category.

```
db.products.aggregate([{$group: {_id: "$category", totalStock: { $sum: "$stock" } } }]);
```

. Find average product price per category.

```
db.products.aggregate([{$group: {_id: "$category", AvgPrice: { $avg: "$price" } } }]);
```

. Find maximum product price.

```
db.products.aggregate([{$group: {_id: "$category", MaxPrice: { $max: "$price" } } }]);
```

. Find minimum product price.

```
db.products.aggregate([{$group: {_id: "$category", MinPrice: { $min: "$price" } } }]);
```

. Calculate total inventory value (price × stock).

```
db.products.aggregate([{$project: { name: 1, inventoryValue: { $multiply: ["$price", "$stock"] } } }]);
```

. Find total quantity ordered per product.

```
db.orders.aggregate([{$group: {_id: "$product_id", totalQty: { $sum: "$quantity" } } }]);
```

. Find total quantity ordered per customer.

```
db.orders.aggregate([{$group: {_id: "$customer_id", totalQty: { $sum: "$quantity" } } }]);
```

\$lookup Queries

. **Display orders with customer details.**

```
db.orders.aggregate([
  {
    $lookup: { from: "customers", localField: "customer_id", foreignField: "customer_id", as: "c" }
  },
  { $unwind: "$c" }
]);
```

. Display orders with product details.

```
db.orders.aggregate([
  {
    $lookup: { from: " products", localField: " product_id", foreignField: "product_id", as: "p" }
  },
  { $unwind: "$p" }
]);
```

. Display customer name and product name per order.

```
db.orders.aggregate([
  {
    $lookup: { from: "customers", localField: "customer_id", foreignField: "customer_id", as: "c" }
  },
  {
    $lookup: { from: "products", localField: "product_id", foreignField: "product_id", as: "p" }
  },
  { $unwind: "$c" },
  { $unwind: "$p" },
  {
    $project: {
      _id: 0,
      order_id: 1,
      customer_name: "$c.name",
      product_name: "$p.name"
    }
  }
]);
```

. Display customer name, city, product name, quantity, and order status.

```
db.orders.aggregate([
  {
    $lookup: { from: "customers", localField: "customer_id", foreignField: "customer_id", as: "c" }
  },
  {
    $lookup: { from: "products", localField: "product_id", foreignField: "product_id", as: "p" }
  },
  { $unwind: "$c" },
  { $unwind: "$p" },
  {
    $project: {
      _id: 0,
```

```
customer_name: "$c.name",  
city: "$c.city",  
product_name: "$p.name",  
quantity: 1,  
status: 1  
}  
}  
));
```
